DATABASE MANAGEMENT SYSTEMS

Residential Society Resident & Maintenance Billing System

COMBINED REVIEW 2 & REVIEW 3

Implementation • Working Prototype • Testing • Final Outcomes

Krish | Roll No. 25WU0102333

Team AIML Rhinos • Project No. 42

Woxsen University | DBMS Project

PROJECT PROGRESSION

From Review 1 design to Review 2 & 3

The initial review established the requirements and database design. This combined review focuses on implementation evidence and prototype results.

REVIEW 1 • DESIGN

Problem statement and objectives
Main users and entities
ER model and relationships
Relational tables and 3NF
Integrity rules and system flow

REVIEW 2 • IMPLEMENT

Python-based browser application
Database tables and relationships
CRUD-style forms and billing logic
Payment tracking and complaints
SQL schema and constraints

REVIEW 3 • VALIDATE

Demonstrate end-to-end workflows
Review reports and status updates
Run smoke tests on core functions
Discuss outcomes and limitations
Show reproducible project files

Goal: demonstrate a working academic prototype built from the Review 1 database design.

System architecture and implementation scope

The local prototype is intentionally lightweight for demonstration; a MySQL schema is provided separately for DBMS implementation discussion.

APPLICATION DATA FLOW

01 USER / SOCIETY ADMIN — Uses forms, lists and reports

02 BROWSER INTERFACE — HTML forms and CSS-styled pages

03 PYTHON APPLICATION — Routes requests, validates inputs and applies billing logic

04 SQLITE DATABASE — Stores prototype records and transaction history

MYSQL REFERENCE SCHEMA

db/schema_mysql.sql contains the relational schema and payment-status trigger logic for DBMS discussion.

PROTOTYPE DATABASE

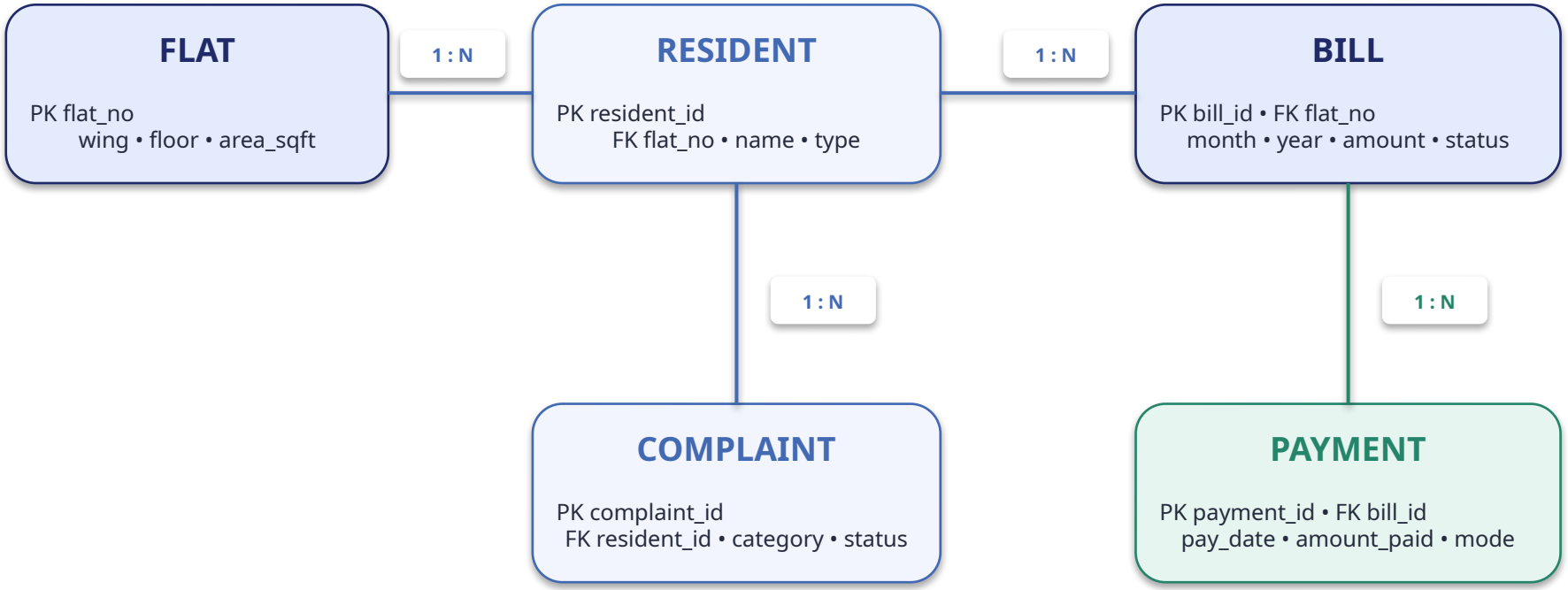
The browser demo runs locally on SQLite so the reviewer can try the workflows without configuring a database server.

Scope distinction SQLite = live demo • MySQL = reference schema

Foreign keys connect residents, flats, bills, payments and complaints; reports query those stored relationships.

Relational model: core entities and relationships

The implementation follows the five core entities described in the original review: flats, residents, maintenance bills, payments and complaints.



Primary keys identify each row; foreign keys connect related records across tables.

Database schema and key fields

A normalized structure avoids repeatedly storing flat or resident details in every transaction.

TABLE	PRIMARY KEY	FOREIGN KEY / IMPORTANT FIELDS
FLAT	flat_no	wing, floor, area_sqft
RESIDENT	resident_id	flat_no → FLAT; name, phone, type
MAINTENANCE_BILL	bill_id	flat_no → FLAT; month, year, amount, due_date, status
PAYMENT	payment_id	bill_id → MAINTENANCE_BILL; pay_date, amount_paid, mode
COMPLAINT	complaint_id	resident_id → RESIDENT; category, date_raised, status

NORMALIZATION

Separate entities reduce duplicate data.

REFERENTIAL INTEGRITY

Related IDs point to existing parent records.

UNIQUE MONTHLY BILL

One bill per flat for each month and year.

Implementation logic and data integrity

Input checks and database constraints protect the core records from common data-entry errors.

DATABASE RULES

- Each flat number is unique.
- Floor and area must be valid values.
- Bills have a valid month/year and non-negative amount.
- A flat cannot receive duplicate bills for the same month/year.
- Payments must be positive and cannot exceed the remaining balance.
- Foreign keys keep payment and complaint references connected.

MAIN APPLICATION OPERATIONS

CREATE	Add flats, residents, bills, payments and complaints
READ	Display registers, statuses and reports
UPDATE	Change complaint status and bill/payment-derived status
REPORT	Calculate unpaid balances and monthly collection totals

PROTOTYPE WALKTHROUGH

Working prototype: society office dashboard

The browser interface groups the implemented operations into clear pages that can be demonstrated during the review.

SocietyDesk / Dashboard

MAIN MENU

- Dashboard
- Flats
- Residents
- Maintenance Bills
- Payments
- Complaints
- Reports

Society overview

FLATS	RESIDENTS	PENDING BILLS	COLLECTED
08	12	05	₹—

Recent billing activity

FLAT	PERIOD	AMOUNT	PAID	STATUS
A-101	10/2026	₹5,000	₹2,000	PARTIAL
A-102	10/2026	₹4,250	₹0	UNPAID

Illustrative screen values are mock presentation examples; open the live application for current sample data.

PROTOTYPE WALKTHROUGH

Workflow 1: register a flat and generate a bill

Monthly bill calculation is driven by the flat area and a configurable demonstration rate.

01

Register flat

Flat no., wing, floor and area in sq. ft.

02

Select period

Month, year and due date

03

Set demo rate

Rate per sq. ft. can be changed

04

Generate

One bill per flat and billing period

DEMONSTRATION EXAMPLE

1,000 sq. ft. × ₹5 per sq. ft. = ₹5,000 bill

₹5 / sq. ft. is a sample rate used for the demo, not an official society rate specified in Review 1.

Workflow 2: payments and complaint tracking

The prototype updates billing status from recorded payments and lets the society office track resident service requests.

PAYMENT LIFECYCLE

UNPAID → PARTIAL → PAID

- Record payment date, amount and method.
- Supports partial or full settlement.
- Paid total and remaining balance are shown.
- Overpayment attempts are rejected.

COMPLAINT TRACKER

OPEN → IN PROGRESS → RESOLVED

- Link each request to a registered resident.
- Capture category, date and a short description.
- Update status as work progresses.
- Use the complaint list to review unresolved items.

Payment methods in the prototype: UPI, cash, card and bank transfer.

Reports: from stored transactions to useful information

Reports use the current database records to help the society office review pending dues and monthly collections.

Outstanding dues

Bills where recorded payments are lower than the billed amount

FLAT	BILLED	PAID	BALANCE	STATUS
A-101	₹5,000	₹2,000	₹3,000	PARTIAL
A-102	₹4,250	₹0	₹4,250	UNPAID
B-204	₹3,500	₹3,500	₹0	PAID

MONTHLY COLLECTION

Total bills, amount collected and collection rate by period.

DASHBOARD SUMMARY

Counts for flats, residents, pending bills and recorded collections.

Illustrative report values shown here explain the layout; the running prototype computes reports from its local demo database.

VALIDATION

Testing and validation results

The bundled smoke-test script was run against the local prototype to verify the core pages and main data operations.

TEST AREA	CHECK PERFORMED	RESULT
Navigation	Dashboard and all module pages return successfully	PASS
Flat registration	New flat record is created and persists	PASS
Monthly billing	Bill amount uses area × configured rate	PASS
Duplicate bill protection	Existing flat/month/year bill is skipped	PASS
Payments	Partial/full payments update bill status	PASS
Overpayment handling	Payment above remaining balance is rejected	PASS
Complaints	New complaint is stored against a resident	PASS

All included smoke tests passed. This is a functional sanity check, not a large-scale or production security test.

Reproducibility: repository structure and run steps

Code, schema files, test script and documentation are organized so the reviewer can reproduce the local demo.

REPOSITORY ROOT

run.py	Prototype entry point
db/schema_sqlite.sql	Local demo schema
db/schema_mysql.sql	MySQL schema reference
tests/smoke_test.py	Automated smoke checks
docs/	Video script and submission checklist
presentations/	Review decks and exports
reports/	Final report draft

RUN THE DEMO

1. Extract repository ZIP
2. Open a terminal in the root folder
3. Run python run.py
4. Open http://127.0.0.1:8000
5. Run python tests/smoke_test.py to recheck

Use fictional sample records during recording.

Results, limitations and future scope

The current prototype demonstrates the core billing workflow while leaving advanced operational features as future development.

ACHIEVED

Centralized registers
Monthly bill calculation
Payment and balance tracking
Complaint status tracking
Pending-dues and collection reports

CURRENT LIMITATIONS

Local academic demo only
Uses SQLite at runtime
No production login / roles
No deployed public service
No payment gateway or backups

FUTURE ENHANCEMENTS

Role-based authentication
Visitor log and facility booking
PDF receipts and reminders
Backup and recovery plan
Secure hosting and MySQL runtime

Visitors and facility bookings appear in the Review 1 conceptual scope but are not implemented in the current runnable prototype.

CONCLUSION

From database design to a working prototype

01 **Normalized relational design**

02 **Working resident and billing workflows**

03 **Payment status and reports**

04 **Smoke-tested and reproducible code**

Thank you

Questions?